Data analytics

A hands-on workshop by Le Wagon

Agenda

- Python 101
- Your first Exploratory Data Analysis with Pandas, matplotlib and Seaborn

We will code in a Notebook

Python 101

Simple Built-in Types

Here are the basic types we need:

- Numeric (`int` and `float`)
- Text (`str`)
- Boolean (`bool`)

Simple Built-in Types

Here are the basic types we need:

- Numeric (`int` and `float`)
- Text (`str`)
- Boolean (`bool`)

Ask yourself: what's the type for a *street number* , a *zip code* , an *annual income* , etc.

Variables

Storing and re-using values in memory:

```
city = "Paris"
```

Functions

Exemple: built-in functions

```
len("Paris")
```

Objects

Stored in **variables**. We can call **methods** (~functions) *on* them.

Example:

```
name = "boris"

 name.capitalize()
```

Objects

Stored in **variables**. We can call **methods** (~functions) *on* them.

Example:

```
name = "boris"

 name.capitalize()
```

Sometimes we call methods *with arguments* :

```
 name.endswith("s")
```

Data Sourcing

The art of *collecting* data priori to an analysis

Finding Data

- I can export it from a software (CSV)
- I know it exists somewhere in a database
- It's on this website I visit daily
- I have found a service (API?) that gives access to it
- I received a file by email

Data is in a file (or many files!)

- File - Structured Data (CSV, JSON, XML or XLSX)
- File - Unstructured Data (PDF, DOCX, TXT or XSLX)
- File - Binary and/or proprietary (How can I open it?)

Data is online

On the Internet or behind company VPN

- It's on a simple **SQL** database and I have access to it
- It's on Google BigQuery or another Data Warehouse provider (SQL usually works too)
- There is an **API** & Documentation
- There is not. **Web Scraping** is the only option

Finding a Dataset

- Google Dataset Search
- Kaggle

Pandas

pandas.pydata.org

Why not Excel?

- Handling **large** datasets (no limit on number of rows)
- Combine multiple files
- Pull data from APIs
- Python!

Pandas IO Tools

Simplest way of using Pandas is to load data directly **from a CSV** file on disk:

```
import pandas as pd


 a_dataframe = pd.read_csv(FILEPATH)
```

Pandas IO Tools

Simplest way of using Pandas is to load data directly **from a CSV** file on disk:

```
import pandas as pd


 a_dataframe = pd.read_csv(FILEPATH)
```

Pandas can also load data from multiple other sources:

👉 pandas.pydata.org/pandas-docs/stable/user_guide/io.html

Seaborn

We will use `seaborn` for **Data Visualization** (it's based on `matplotlib`)

```
import seaborn as sns
```

First, let's choose a dataset to explore

New York City Airbnb Open Data

New York City Airbnb Open Data

Let's use this Dataset from 2019 available on Kaggle:

👉 kaggle.com/dgomonov/new-york-city-airbnb-open-data

Loading the CSV

We can use the `pandas.read_csv` function.

```
listings_df = pd.read_csv('data/AB_NYC_2019.csv')
```

We now have `listings_df`, a variable containing our first **DataFrame**

Quick look

Let's use some attributes & methods of the DataFrame:

- `.shape`
- `.columns`
- `.dtypes`
- `.head()` / `tail()`

Cleaning Data

We can show how many nulls are found in each column with:

```
listings_df.isnull().sum()
```

Cleaning Data

We can show how many nulls are found in each column with:

```
listings_df.isnull().sum()
```

We can drop some columns we find not relevant enough for our analysis:

```
columns_to_drop = ['id', 'host_name', 'last_review']
listings_df.drop(columns_to_drop, axis="columns", inplace=True)
```

Cleaning Data

We can show how many nulls are found in each column with:

```
listings_df.isnull().sum()
```

We can drop some columns we find not relevant enough for our analysis:

```
columns_to_drop = ['id', 'host_name', 'last_review']
listings_df.drop(columns_to_drop, axis="columns", inplace=True)
```

We can replace NaN with:

```
listings_df.fillna({'reviews_per_month': 0}, inplace=True)
```

Columns & Rows

Columns & Rows

We can filter **columns** (by name):

```
listings_df['name'] # => Series

# or

 listings_df[['name', 'neighbourhood_group', 'price']] # => DataFrame
```

Columns & Rows

We can filter **columns** (by name):

```
listings_df['name'] # => Series

# or

 listings_df[['name', 'neighbourhood_group', 'price']] # => DataFrame
```

Or **rows** (by index):

```
 listings_df[5:10]
```

Columns & Rows

We can filter **columns** (by name):

```
listings_df['name'] # => Series

# or

 listings_df[['name', 'neighbourhood_group', 'price']] # => DataFrame
```

Or **rows** (by index):

```
 listings_df[5:10]
```

And even **combine** both approaches

Boolean Indexing

Boolean Indexing

```
listings_df['price'] < 100
```

Boolean Indexing

```
listings_df['price'] < 100
```

Using this Series as an selection index:

```
listings_df[listings_df['price'] < 100]
```

Exploratory Data Analysis

Quoting Wikipedia:

> *In statistics, exploratory data analysis (EDA) is an approach to analyzing data sets to summarize their main characteristics, often with visual methods.*

Let's start asking some **questions** 🤔 to our dataset

🤔 What are the 10 most reviewed listings?

🤔 What are the 10 most reviewed listings?

```
listings_df.nlargest(10, 'number_of_reviews')
```

🤔 What are the NY neighourhood groups with listings?

🤔 What are the NY neighourhood groups with listings?

```
listings_df['neighbourhood_group'].unique()
```

🤔 Follow-up: How many listings per neighbourhood group?

🤔 Follow-up: How many listings per neighbourhood group?

```
listings_df['neighbourhood_group'].value_counts()
```

🤔 What are the Top 10 neighbourhoods with Airbnb listings?

🤔 What are the Top 10 neighbourhoods with Airbnb listings?

```
listings_df['neighbourhood'].value_counts().head(10)
```

We can plot a barchart with `matplotlib` calling `.plot(kind='bar')` directly *on* the Dataframe

```
listings_df['neighbourhood'].value_counts().head(10).plot(kind='bar')
```

We can plot a barchart with `matplotlib` calling `.plot(kind='bar')` directly *on* the Dataframe

```
listings_df['neighbourhood'].value_counts().head(10).plot(kind='bar')
```

No need for a `value_counts()` with `seaborn`, we can just use a Countplot

```
sns.countplot(data=listings_df, x='neighbourhood_group')
```

We can plot a barchart with `matplotlib` calling `.plot(kind='bar')` directly *on* the Dataframe

```
listings_df['neighbourhood'].value_counts().head(10).plot(kind='bar')
```

No need for a `value_counts()` with `seaborn`, we can just use a Countplot

```
sns.countplot(data=listings_df, x='neighbourhood_group')
```

```
order = listings_df['neighbourhood_group'].value_counts().index
sns.countplot(data=listings_df, x='neighbourhood_group', order=order)
```

🤔 What's the influence neighbourhood group on **room type**?

FYI, here are the room types:

```
listings_df['room_type'].unique()
```

🤔 What's the influence neighbourhood group on **room type**?

FYI, here are the room types:

```
listings_df['room_type'].unique()
```

```
sns.countplot(data=listings_df, x="neighbourhood_group", hue="room_type")
```

🤔 What is the **distribution** of listing prices?

🤔 What is the **distribution** of listing prices?

We can use a Distplot:

```
sns.distplot(listings_df['price'])
```

🤔 What is the **distribution** of listing prices?

We can use a Distplot:

```
sns.distplot(listings_df['price'])
```

How many listings are **more expensive than 500$/night**? Could we remove those outliers from the visualization? What technique can we use?

🤔 What is the **distribution** of listing prices?

We can use a Distplot:

```
sns.distplot(listings_df['price'])
```

How many listings are **more expensive than 500$/night**? Could we remove those outliers from the visualization? What technique can we use?

```
affordable_df = listings_df[listings_df['price'] <= 500]
sns.distplot(affordable_df['price'])
```

🤔 What is the **distribution** of listing prices?

We can use a Distplot:

```
sns.distplot(listings_df['price'])
```

How many listings are **more expensive than 500$/night**? Could we remove those outliers from the visualization? What technique can we use?

```
affordable_df = listings_df[listings_df['price']<= 500]
sns.distplot(affordable_df['price'])
```

We can ask Pandas for the mean:

```
listings_df.price.mean()
affordable_df.price.mean()
```

🤔 What is the **distribution** of listing prices based on the Neighbourhood Group?

🤔 What is the **distribution** of listing prices based on the Neighbourhood Group?

We can use a Violinplot:

```
sns.violinplot(data=affordable_df, x="neighbourhood_group", y="price")
```

🤔 What is the **distribution** of listing prices based on the Neighbourhood Group?

We can use a Violinplot:

```
sns.violinplot(data=affordable_df, x="neighbourhood_group", y="price")
```

We can make the figure bigger calling this before the `sns.violinplot(...)`

```
from matplotlib import pyplot as plt


plt.figure(figsize=(15, 8))
```

🤔 Can we plot the listings on a map?

🤔 Can we plot the listings on a map?

Back to `matplotlib`, we can use a **scatterplot**:

```
affordable_df.plot(
    kind='scatter',
    x='longitude',
    y='latitude',
    c='price',
    cmap='inferno',
    colorbar=True,
    alpha=0.8,
      figsize=(12,8))
```

Bonus: Adding NYC map underneath!

```
background_image =
plt.imread('https://raw.githubusercontent.com/lewagon/data-images/master/workshops/Neighbourhoods_New_York_City_Map.png')
plt.imshow(background_image, zorder=0, extent=[-74.258, -73.7, 40.49, 40.92])

ax = plt.gca()
affordable_df.plot(
  ax=ax,
  zorder=1,
  kind='scatter',
  x='longitude',
  y='latitude',
  c='price',
  cmap='inferno',
  colorbar=True,
  alpha=0.8,
  figsize=(12,8)
)
```

Going Further

Some topics for another time

SQL

You can't be *too* good at SQL!

```
SELECT
    ROUND(EXTRACT(epoch FROM age(_camp_starts_at, birthday)) / (3600 * 24 * 365)) as age,
    COUNT(DISTINCT id)
FROM alumni
GROUP BY 1
HAVING ROUND(EXTRACT(epoch FROM age(_camp_starts_at, birthday)) / (3600 * 24 * 365)) > 15
AND ROUND(EXTRACT(epoch FROM age(_camp_starts_at, birthday)) / (3600 * 24 * 365)) < 90
 ORDER BY 1
```

API Fetching

You need the `requests` package.

```
import requests


url = "https://..."
response = requests.get(url) # `GET` HTTP request
print(response.status_code)  # Should be 200 if OK
data = response.json()


 # TODO: Load `data` in a Pandas DataFrame
```

Web Scraping

You need the `BeautifulSoup` package and some front-end development knowledge (DOM, CSS selectors, etc.)

```
import requests

from bs4 import BeautifulSoup


response = requests.get(url)

soup = BeautifulSoup(response.content, "html.parser")


# You now can query the `soup` object!

soup.title.string

soup.find('h1')

soup.find_all('a')

  # etc...
```

Machine Learning

Do your first regressions, classification & clustering with the `scikit-learn` package.

Your turn!